



Universidade Federal de Goiás  
Instituto de Matemática e Estatística  
Bacharelado em Matemática Aplicada e Computacional

Componente Curricular: INF0286 - ALGORITMOS E ESTRUTURAS DE DADOS 1  
Turma A: 2025/2  
Professora: Dra. Renata Dutra Braga

### Lista 05 (23/09/25)

1) Implemente uma pilha de inteiros com vetor de tamanho fixo (MAX). Crie as funções `init`, `empty`, `full`, `push`, `pop`. No `main`, empilhe 5 notas (ex.: 7, 8, 9, 6, 10) e depois desempilhe tudo, mostrando na tela para evidenciar a ordem LIFO.

Sugestões de técnicas (C) e estrutura.

- `typedef struct` com `int v[MAX]` e `int topo`.
- Checagem de limites (under/overflow).
- Bibliotecas: `<stdio.h>`, `<stdbool.h>`.
- Estrutura: pilha estática (vetor fixo).

```
#include <stdio.h>
#include <stdbool.h>
#define MAX 100

typedef struct { int v[MAX]; int topo; } Stack;

void init(Stack *s){ s->topo = -1; }
bool empty(Stack *s){ return s->topo == -1; }
bool full(Stack *s){ return s->topo == MAX - 1; }

bool push(Stack *s, int x){
    if (full(s)) return false;
    s->v[++s->topo] = x;
    return true;
}

bool pop(Stack *s, int *out){
    if (empty(s)) return false;
    *out = s->v[s->topo--];
    return true;
}
```

```

int main(void){
    Stack s; init(&s);
    int vals[5] = {7,8,9,6,10}, x;
    for (int i=0;i<5;i++) push(&s, vals[i]);
    while (pop(&s,&x)) printf("%d ", x);
    puts("");
    return 0;
}

```

2) Reaproveite a pilha do exercício 1 e acrescente a função `top` que consulta o elemento do topo sem removê-lo. No `main`, empilhe três valores e mostre o topo, sem alterar a pilha.

Sugestões de técnicas (C) e estrutura.

- Mesmo struct do ex. 1; função `top(Stack*, int*)`.
- Estrutura: pilha estática.

```

#include <stdio.h>
#include <stdbool.h>
#define MAX 100

typedef struct { int v[MAX]; int topo; } Stack;

void init(Stack *s){ s->topo=-1; }
bool empty(Stack *s){ return s->topo==-1; }
bool full(Stack *s){ return s->topo==MAX-1; }
bool push(Stack *s,int x){ if(full(s)) return false; s->v[++s->topo]=x; return true; }
bool top(Stack *s,int *out){ if(empty(s)) return false; *out=s->v[s->topo]; return true; }

int main(void){
    Stack s; init(&s);
    push(&s,10); push(&s,20); push(&s,30);
    int t;
    if (top(&s,&t)) printf("Topo=%d\n", t); else puts("Pilha vazia");
    return 0;
}

```

3) Leia uma palavra (sem espaços) e imprima-a invertida usando uma pilha de char. O código deve empilhar cada caractere e depois desempilhar para formar a inversão.

Sugestões de técnicas (C) e estrutura.

- `char v[MAX]` na pilha; leitura com `scanf("%s", ...)`.
- Estrutura: pilha estática de char.

```

#include <stdio.h>
#include <string.h>
#include <stdbool.h>
#define MAX 256

typedef struct { char v[MAX]; int topo; } Stack;

void init(Stack *s){ s->topo=-1; }
bool empty(Stack *s){ return s->topo== -1; }
bool full(Stack *s){ return s->topo==MAX-1; }
bool push(Stack *s,char c){ if(full(s)) return false; s->v[++s->topo]=c; return true; }
bool pop(Stack *s,char *out){ if(empty(s)) return false; *out=s->v[s->topo--]; return true; }

int main(void){
    char word[MAX]; Stack st; init(&st);
    if (scanf("%255s", word)!=1) return 0;
    for (int i=0; word[i]; i++) push(&st, word[i]);
    char c;
    while (pop(&st,&c)) putchar(c);
    putchar('\n');
    return 0;
}

```

4) Simule a sequência de chamadas: main -> f1 -> f2. Empilhe os nomes das funções conforme “entram” e desempilhe ao “retornar”, imprimindo mensagens que mostrem a ordem LIFO.

Sugestões de técnicas (C) e estrutura.

- Vetor de char [ SIZE ] [ LEN ] para armazenar nomes.
- Estrutura: pilha estática de strings.

```

#include <stdio.h>
#include <string.h>
#include <stdbool.h>
#define MAX 10
#define LEN 32

typedef struct { char v[MAX][LEN]; int topo; } CallStack;
void init(CallStack *s){ s->topo=-1; }
bool push(CallStack*s,const char*name){ if(s->topo==MAX-1) return false;
strcpy(s->v[++s->topo],name); return true; }
bool pop(CallStack*s,char*out){ if(s->topo== -1) return false; strcpy(out,s->v[s->topo--]);
return true; }

int main(void){
    CallStack cs; init(&cs); char out[LEN];
    puts("Entrando em main"); push(&cs,"main");
    puts("Chamando f1");      push(&cs,"f1");
    puts("Chamando f2");      push(&cs,"f2");
}

```

```
while(pop(&cs,out)) printf("Retornando de %s\n", out);  
return 0;  
}
```

5) Implemente uma fila circular de nomes (char[LEN]). Funções: init, empty, full, enqueue, dequeue, front. Enfileire “Ana”, “Beto”, “Clara” e atenda na ordem (FIFO).

Sugestões de técnicas (C) e estrutura.

- Índices in, out e contador cnt.
- Operador módulo %N.
- Estrutura: fila circular estática de strings

```
#include <stdio.h>  
#include <string.h>  
#include <stdbool.h>  
#define N 5  
#define LEN 32  
  
typedef struct { char q[N][LEN]; int in,out,cnt; } Queue;  
  
void init(Queue*Q){ Q->in=Q->out=Q->cnt=0; }  
bool empty(Queue*Q){ return Q->cnt==0; }  
bool full(Queue*Q){ return Q->cnt==N; }  
bool enqueue(Queue*Q,const char*s){  
    if(full(Q)) return false;  
    strncpy(Q->q[Q->in], s, LEN-1); Q->q[Q->in][LEN-1]='\0';  
    Q->in=(Q->in+1)%N; Q->cnt++; return true;  
}  
bool dequeue(Queue*Q,char*out){  
    if(empty(Q)) return false;  
    strncpy(out, Q->q[Q->out], LEN); Q->out=(Q->out+1)%N; Q->cnt--; return true;  
}  
bool front(Queue*Q,char*out){  
    if(empty(Q)) return false;  
    strncpy(out, Q->q[Q->out], LEN); return true;  
}  
  
int main(void){  
    Queue Q; init(&Q); char buf[LEN];  
    enqueue(&Q,"Ana"); enqueue(&Q,"Beto"); enqueue(&Q,"Clara");  
    while (dequeue(&Q,buf)) puts(buf);  
    return 0;  
}
```

6) Com a mesma fila do exercício 5, enfileire três nomes e utilize front para consultar quem é o próximo a ser atendido, sem removê-lo. Mostre o nome e depois atenda todos.

Sugestões de técnicas (C) e estrutura.

- Reutilize front + dequeue.
- Estrutura: fila circular estática

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>
#define N 5
#define LEN 32

typedef struct { char q[N][LEN]; int in,out,cnt; } Queue;
void init(Queue*Q){ Q->in=Q->out=Q->cnt=0; }
bool empty(Queue*Q){ return Q->cnt==0; }
bool full(Queue*Q){ return Q->cnt==N; }
bool enqueue(Queue*Q,const char*s){ if(full(Q)) return false;
strncpy(Q->q[Q->in],s,LEN-1); Q->q[Q->in][LEN-1]='\0'; Q->in=(Q->in+1)%N; Q->cnt++;
return true; }
bool dequeue(Queue*Q,char*out){ if(empty(Q)) return false;
strncpy(out,Q->q[Q->out],LEN); Q->out=(Q->out+1)%N; Q->cnt--; return true; }
bool front(Queue*Q,char*out){ if(empty(Q)) return false; strncpy(out,Q->q[Q->out],LEN);
return true; }

int main(void){
    Queue Q; init(&Q); char buf[LEN];
    enqueue(&Q,"Ana"); enqueue(&Q,"Beto"); enqueue(&Q,"Clara");
    if (front(&Q,buf)) printf("Proximo: %s\n", buf);
    while (dequeue(&Q,buf)) printf("Atendido: %s\n", buf);
    return 0;
}
```

7) Simule uma fila de impressão: enfileire 4 nomes de arquivos (“a.pdf”, “b.doc”, ...) e desenfileire todos, imprimindo a ordem de processamento.

Sugestões de técnicas (C) e estrutura.

- Mesmo TAD do ex. 5; apenas nomes diferentes.
- Estrutura: fila circular estática

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>
#define N 6
#define LEN 40

typedef struct { char q[N][LEN]; int in,out,cnt; } Queue;
void init(Queue*Q){ Q->in=Q->out=Q->cnt=0; }
bool empty(Queue*Q){ return Q->cnt==0; }
bool full(Queue*Q){ return Q->cnt==N; }
bool enqueue(Queue*Q,const char*s){ if(full(Q)) return false;
```

```

snprintf(Q->q[Q->in],LEN,"%s",s); Q->in=(Q->in+1)%N; Q->cnt++; return true; }
bool dequeue(Queue*Q,char*out){ if(empty(Q)) return false;
snprintf(out,LEN,"%s",Q->q[Q->out]); Q->out=(Q->out+1)%N; Q->cnt--; return true; }

int main(void){
    Queue Q; init(&Q); char out[LEN];
    enqueue(&Q,"a.pdf"); enqueue(&Q,"b.doc"); enqueue(&Q,"c.png"); enqueue(&Q,"d.txt");
    puts("Processando fila de impressao:");
    while (dequeue(&Q,out)) puts(out);
    return 0;
}

```

8) Coloque 5 alunos em uma fila. Atenda 10 passos: a cada passo, pegue o primeiro, imprima "Atendido: X" e reenfileire o mesmo nome no final (simulando rodízio). Mostre a sequência produzida.

Sugestões de técnicas (C) e estrutura.

- Reutilize dequeue seguido de enqueue.
- Estrutura: fila circular estática.

```

#include <stdio.h>
#include <string.h>
#include <stdbool.h>
#define N 8
#define LEN 32

typedef struct { char q[N][LEN]; int in,out,cnt; } Queue;
void init(Queue*Q){ Q->in=Q->out=Q->cnt=0; }
bool empty(Queue*Q){ return Q->cnt==0; }
bool full(Queue*Q){ return Q->cnt==N; }
bool enqueue(Queue*Q,const char*s){ if(full(Q)) return false;
snprintf(Q->q[Q->in],LEN,"%s",s); Q->in=(Q->in+1)%N; Q->cnt++; return true; }
bool dequeue(Queue*Q,char*out){ if(empty(Q)) return false;
snprintf(out,LEN,"%s",Q->q[Q->out]); Q->out=(Q->out+1)%N; Q->cnt--; return true; }

int main(void){
    Queue Q; init(&Q); char buf[LEN];
    const char* nomes[5]={"Ana","Beto","Clara","Davi","Eva"};
    for(int i=0;i<5;i++) enqueue(&Q,nomes[i]);

    for(int step=1; step<=10; step++){
        if (dequeue(&Q,buf)){
            printf("Atendido: %s\n", buf);
            enqueue(&Q, buf); // volta pro fim
        }
    }
    return 0;
}

```

9) Implemente uma lista sequencial (vetor fixo) de int com size atual. Funções: push\_back, print, remove\_first(value) (remove a primeira ocorrência, deslocando elementos). Demonstre inserindo 1,2,3 e removendo 2.

Sugestões de técnicas (C) e estrutura.

- Vetor int a[MAX] + int size.
- Deslocamento com laço for.
- Estrutura: lista sequencial estática.

```
#include <stdio.h>
#define MAX 100

typedef struct { int a[MAX]; int size; } AList;

void init(AList*L){ L->size=0; }
int push_back(AList*L,int x){ if(L->size==MAX) return 0; L->a[L->size++]=x; return 1; }
int remove_first(AList*L,int x){
    int pos=-1;
    for(int i=0;i<L->size;i++) if(L->a[i]==x){ pos=i; break; }
    if(pos==-1) return 0;
    for(int i=pos;i<L->size-1;i++) L->a[i]=L->a[i+1];
    L->size--; return 1;
}
void print(const AList*L){ for(int i=0;i<L->size;i++) printf("%d ",L->a[i]); puts(""); }

int main(void){
    AList L; init(&L);
    push_back(&L,1); push_back(&L,2); push_back(&L,3);
    print(&L);
    remove_first(&L,2);
    print(&L);
    return 0;
}
```

10) Crie uma lista simplesmente encadeada de int com insert\_front e print. Insira na ordem: 5, depois 7, depois 9 (sempre no início) e mostre a lista resultante.

Sugestões de técnicas (C) e estrutura.

- typedef struct Node { int v; struct Node\* next; }.
- malloc/free quando necessário.
- Estrutura: lista encadeada simples.

```
#include <stdio.h>
#include <stdlib.h>
```

```

typedef struct Node { int v; struct Node* next; } Node;

void insert_front(Node**h,int x){
    Node* n = (Node*)malloc(sizeof(Node));
    n->v=x; n->next=*h; *h=n;
}
void print(Node*h){ for(;h;h=h->next) printf("%d ",h->v); puts(""); }
void free_all(Node*h){ while(h){ Node* t=h; h=h->next; free(t);} }

int main(void){
    Node* head=NULL;
    insert_front(&head,5);
    insert_front(&head,7);
    insert_front(&head,9);
    print(head);
    free_all(head);
    return 0;
}

```

11) Na lista do exercício 10, implemente `find(Node* h, int x)` que retorna ponteiro para o nó com valor `x` (ou `NULL` se não existir). Use para buscar o valor 7 e imprima uma mensagem adequada.

Sugestões de técnicas (C) e estrutura.

- Percurso com `for/while`.
- Estrutura: lista encadeada simples.

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int v; struct Node* next; } Node;

void insert_front(Node**h,int x){
    Node* n=(Node*)malloc(sizeof(Node));
    n->v=x; n->next=*h; *h=n;
}
Node* find(Node* h,int x){
    for(;h;h=h->next) if(h->v==x) return h;
    return NULL;
}
void free_all(Node*h){ while(h){ Node*t=h; h=h->next; free(t);} }

int main(void){
    Node* head=NULL;
    insert_front(&head,5); insert_front(&head,7); insert_front(&head,9);
    Node* p = find(head,7);
    puts(p? "7 encontrado":"7 nao encontrado");
}

```



```
    free_all(head);  
    return 0;  
}
```

12) Implemente `remove_first(Node** h, int x)` que remove a primeira ocorrência de `x` da lista (ajustando ponteiros corretamente). Teste removendo 7 da lista criada anteriormente.

Sugestões de técnicas (C) e estrutura.

- Manter ponteiros `prev` e `curr`.
- Tratar o caso de remoção na cabeça.
- Estrutura: lista encadeada simples.

```
#include <stdio.h>  
#include <stdlib.h>  
  
typedef struct Node { int v; struct Node* next; } Node;  
  
void insert_front(Node**h,int x){  
    Node*n=(Node*)malloc(sizeof(Node));  
    n->v=x; n->next=*h; *h=n;  
}  
  
int remove_first(Node**h,int x){  
    Node *prev=NULL, *cur=*h;  
    while(cur && cur->v!=x){ prev=cur; cur=cur->next; }  
    if(!cur) return 0;  
    if(prev) prev->next=cur->next; else *h=cur->next;  
    free(cur); return 1;  
}  
  
void print(Node*h){ for(;h;h=h->next) printf("%d ",h->v); puts(""); }  
void free_all(Node*h){ while(h){ Node*t=h; h=h->next; free(t);} }  
  
int main(void){  
    Node* head=NULL;  
    insert_front(&head,5); insert_front(&head,7); insert_front(&head,9);  
    print(head);  
    if (remove_first(&head,7)) puts("Removido 7"); else puts("7 nao encontrado");  
    print(head);  
    free_all(head);  
    return 0;  
}
```